

# Detailed Implementation and Publication Roadmap

## Risk-Triggered Adaptive MPC Safety Filtering with LQR

### Nominal Control for Differential-Drive Robot Navigation

Your Name

May 23, 2026

## 1 Purpose of This Document

This document gives a step-by-step implementation roadmap for building, testing, evaluating, and writing a research paper on:

### **Risk-Triggered Adaptive MPC Safety Filtering with LQR Nominal Control for Differential-Drive Robot Navigation**

The goal is to convert the current project into a publication-oriented research work. The method should not be presented as random switching or blending between LQR and MPC. Instead, the final controller should be presented as:

LQR is the nominal tracking controller. Adaptive MPC is a safety filter that activates only when the predicted LQR trajectory becomes unsafe.

## 2 Final Controller Idea

At every control step, the robot performs the following logic:

1. Read robot state from odometry.
2. Read obstacle information from Gazebo obstacle publisher or LiDAR.
3. Update actuation mismatch parameters using LMS.
4. Compute nominal LQR command.
5. Predict the future trajectory if only LQR is applied.
6. If the predicted LQR rollout is safe, apply LQR.
7. If the predicted LQR rollout is unsafe, activate adaptive MPC safety filter.
8. If adaptive MPC fails or is too slow, activate backup safety controller.

The high-level algorithm is:

```
x = get_current_state()
obstacles = get_obstacles()
theta_hat = lms.update(x_prev, u_prev, x)

u_lqr = lqr.compute_control(x, x_ref, u_ref, theta_hat)
```

```

predicted_lqr_path = rollout(x, u_lqr, theta_hat)

if is_safe(predicted_lqr_path, obstacles):
    u_cmd = u_lqr
    mode = "LQR"
else:
    sol = adaptive_mpc.solve_safety_filter(
        x0=x,
        u_nominal=u_lqr,
        theta_hat=theta_hat,
        obstacles=obstacles
    )

    if sol.success:
        u_cmd = sol.u0
        mode = "ADAPTIVE_MPC_FILTER"
    else:
        u_cmd = backup_controller(x, obstacles)
        mode = "BACKUP"

```

### 3 Important Warning

Do not claim that publication is guaranteed. The correct goal is to make the work scientifically defensible. The paper becomes stronger if the implementation contains:

- proper baselines,
- realistic Gazebo mismatch,
- clear metrics,
- good plots,
- honest limitations.

### 4 Recommended Project Structure

Inside the existing VS Code project, create or modify the following files:

| File                                       | Purpose                                                               |
|--------------------------------------------|-----------------------------------------------------------------------|
| controllers/lms_adaptation.py              | LMS parameter estimator for velocity and angular velocity scale.      |
| controllers/risk_predictor.py              | Rolls out LQR command and checks future obstacle risk.                |
| controllers/adaptive_safety_filter_node.py | Main proposed controller wrapper.                                     |
| controllers/backup_safety.py               | Emergency braking or CBF-style backup controller.                     |
| nodes/adaptive_safety_filter_node.py       | ROS2 node that runs the proposed controller.                          |
| nodes/command_distortion_node.py           | Injects actuation mismatch in Gazebo.                                 |
| evaluation/run_proposed_experiments.py     | Runs all baseline and proposed experiments.                           |
| evaluation/metrics.py                      | Computes RMSE, collision count, solver time, jerk, intervention rate. |
| evaluation/plot_results.py                 | Generates publication plots.                                          |

## 5 Phase 1: Freeze the Current Baselines

Before implementing the new method, run and save results from existing controllers.

### 5.1 Controllers to Test

1. LQR only.
2. MPC only.
3. Adaptive MPC only.
4. Existing blended hybrid controller.

### 5.2 Why This Is Needed

These results will become the baseline table in the paper. Without baselines, reviewers cannot know whether the proposed method is actually better.

### 5.3 What to Save

For each run, save:

- robot trajectory,
- reference trajectory,
- control inputs,
- solver time,
- collision count,
- minimum obstacle clearance,
- tracking error.

## 6 Phase 2: Implement LMS Adaptation

The adaptive model is:

$$\dot{p}_x = \hat{v}_v v \cos \theta, \quad (1)$$

$$\dot{p}_y = \hat{v}_v v \sin \theta, \quad (2)$$

$$\dot{\theta} = \hat{v}_\omega \omega. \quad (3)$$

The LMS update is:

$$\hat{v}_{k+1} = \Pi_{\Theta} \left( \hat{v}_k + \Gamma \Phi_k^T e_k \right). \quad (4)$$

The sensitivity matrix is:

$$\Phi_k = \Delta t \begin{bmatrix} v_k \cos \theta_k & 0 \\ v_k \sin \theta_k & 0 \\ 0 & \omega_k \end{bmatrix}. \quad (5)$$

## 6.1 Implementation Rule

Do not update a parameter if the corresponding input is too small:

```
if abs(v) > eps_v:
    theta_v = theta_v + update_v

if abs(omega) > eps_omega:
    theta_omega = theta_omega + update_omega
```

This handles the persistent excitation issue.

## 7 Phase 3: Inject Actuation Mismatch in Gazebo

Gazebo is usually too perfect. To prove that LMS adaptation is useful, create a command distortion node.

The controller outputs:

$$u_c = [v_c \quad \omega_c]^T. \quad (6)$$

The robot receives:

$$u_a = [\lambda_v(k)v_c \quad \lambda_\omega(k)\omega_c]^T. \quad (7)$$

Example mismatch profiles:

1. Constant mismatch:

$$\lambda_v = 0.80, \quad \lambda_\omega = 0.85.$$

2. Step mismatch:

$$\lambda_v = \begin{cases} 1.0, & t < 20s \\ 0.75, & t \geq 20s \end{cases}$$

3. Sinusoidal mismatch:

$$\lambda_v(t) = 0.85 + 0.10 \sin(0.2t).$$

## 8 Phase 4: Implement LQR Nominal Tracking

Use the existing LQR controller as the nominal tracker. The nominal command is:

$$u_{LQR,k} = u_{ref,k} - K_k(x_k - x_{ref,k}). \quad (8)$$

The LQR controller should be active most of the time. This is important because the paper claims reduced computational cost compared to continuous MPC.

## 9 Phase 5: Implement LQR Rollout Risk Predictor

Before applying the LQR command, predict the future trajectory:

$$\hat{x}_{i+1} = f(\hat{x}_i, u_{LQR,i}, \hat{v}_k). \quad (9)$$

For obstacle  $j$ :

$$h_j(x) = \|p - p_{obs,j}\|_2 - (r_{obs,j} + d_{safe}). \quad (10)$$

The rollout is safe if:

$$h_j(\hat{x}_i) \geq 0, \quad \forall i, j. \quad (11)$$

## 9.1 Simple Implementation

```
def is_safe(predicted_states, obstacles, d_safe):
    for x in predicted_states:
        px, py = x[0], x[1]
        for obs in obstacles:
            dist = sqrt((px - obs.x)**2 + (py - obs.y)**2)
            clearance = dist - obs.radius
            if clearance < d_safe:
                return False
    return True
```

## 10 Phase 6: Implement Adaptive MPC Safety Filter

The safety filter should not replace LQR unnecessarily. It should minimize deviation from LQR:

$$\min_{\mathbf{x}, \mathbf{u}} \sum_{i=0}^{N-1} (\|x_i - x_{ref,i}\|_Q^2 + \|u_i - u_{LQR,i}\|_R^2 + \|\Delta u_i\|_S^2) + \|x_N - x_{ref,N}\|_P^2. \quad (12)$$

Subject to:

$$x_0 = x_k, \quad (13)$$

$$x_{i+1} = f(x_i, u_i, \hat{\vartheta}_k), \quad (14)$$

$$u_{min} \leq u_i \leq u_{max}, \quad (15)$$

$$\|p_i - p_{obs,j}\|_2 \geq r_{obs,j} + d_{safe}. \quad (16)$$

### 10.1 Important Coding Detail

Your existing adaptive MPC already has nonlinear dynamics and obstacle constraints. The main change is to add the term:

$$\|u_i - u_{LQR,i}\|_R^2. \quad (17)$$

This makes the MPC act like a safety filter.

## 11 Phase 7: Implement Backup Safety Controller

MPC may fail if:

- an obstacle appears too close,
- actuator limits make avoidance impossible,
- LMS parameters change suddenly,
- the solver exceeds the real-time limit.

A simple backup controller is:

$$v = 0, \quad \omega = k_\omega \cdot \text{sign}(\theta_{away} - \theta). \quad (18)$$

This stops forward motion and rotates away from the nearest obstacle.

## 12 Phase 8: Add LiDAR or Obstacle Input

The controller does not require LiDAR mathematically. It only requires obstacle information.

### 12.1 Simple Version

Use the existing Gazebo obstacle publisher:

$$obstacles = [(x_1, y_1, r_1), (x_2, y_2, r_2), \dots]. \quad (19)$$

### 12.2 Stronger Version

Use simulated LiDAR:

1. Subscribe to `/scan`.
2. Convert LaserScan ranges to points.
3. Cluster nearby points.
4. Approximate each cluster as a circular obstacle.
5. Pass obstacles to the risk predictor and MPC.

For the first publication attempt, the simple obstacle publisher is acceptable if explained honestly.

## 13 Phase 9: Build the Proposed ROS2 Node

Create a new ROS2 node:

`adaptive_safety_filter_node.py`

The node should:

1. subscribe to odometry,
2. subscribe to obstacles or LiDAR,
3. compute reference trajectory,
4. run LMS update,
5. compute LQR command,
6. run rollout risk predictor,
7. run adaptive MPC only if needed,
8. publish `/cmd_vel`,
9. log mode, solver time, control, state, and clearance.

## 14 Phase 10: Logging

Every timestep should save:

| Logged value                                | Purpose                    |
|---------------------------------------------|----------------------------|
| $t$                                         | time axis for plots        |
| $x, y, \theta$                              | robot state                |
| $x_{ref}, y_{ref}, \theta_{ref}$            | reference state            |
| $v, \omega$                                 | applied command            |
| mode                                        | LQR, MPC filter, or backup |
| solver time                                 | computational cost         |
| minimum clearance                           | safety metric              |
| tracking error                              | accuracy metric            |
| $\hat{\vartheta}_v, \hat{\vartheta}_\omega$ | LMS adaptation behavior    |
| collision flag                              | safety result              |

## 15 Phase 11: Evaluation Scenarios

Use at least the following scenarios:

1. No obstacle, no mismatch.
2. Static obstacles, no mismatch.
3. Static obstacles with actuation mismatch.
4. Static obstacles with sensor noise.
5. Dynamic obstacle crossing the robot path.
6. Dense obstacle field.

## 16 Phase 12: Metrics

Tracking RMSE:

$$RMSE = \sqrt{\frac{1}{T} \sum_{k=1}^T ((p_{x,k} - p_{x,ref,k})^2 + (p_{y,k} - p_{y,ref,k})^2)}. \quad (20)$$

Minimum clearance:

$$d_{min} = \min_{k,j} (\|p_k - p_{obs,j}\|_2 - r_{obs,j}). \quad (21)$$

Intervention rate:

$$\rho_{MPC} = \frac{N_{MPC}}{N_{total}}. \quad (22)$$

Linear jerk:

$$j_v(k) = \frac{v_k - 2v_{k-1} + v_{k-2}}{\Delta t^2}. \quad (23)$$

Angular jerk:

$$j_{\omega}(k) = \frac{\omega_k - 2\omega_{k-1} + \omega_{k-2}}{\Delta t^2}. \quad (24)$$

RMS jerk:

$$J_{rms} = \sqrt{\frac{1}{T-2} \sum_{k=3}^T (j_v(k)^2 + j_{\omega}(k)^2)}. \quad (25)$$

## 17 Phase 13: Required Plots

The paper should contain:

1. Trajectory plot with obstacles.
2. Intervention map:
  - green = LQR active,
  - yellow = adaptive MPC filter active,
  - red = backup active.
3. Solver time plot.
4. Tracking error plot.
5. Control input plot.
6. LMS parameter estimate plot.
7. Bar chart comparing collision count and RMSE across baselines.

## 18 Phase 14: Results Table Template

Use a table like this in the paper:

Table 3: Controller comparison across obstacle and mismatch scenarios.

| Controller      | RMSE | Collisions | $d_{min}$ | Time | Jerk |
|-----------------|------|------------|-----------|------|------|
| LQR             | —    | —          | —         | —    | —    |
| MPC             | —    | —          | —         | —    | —    |
| Adaptive MPC    | —    | —          | —         | —    | —    |
| Blended LQR-MPC | —    | —          | —         | —    | —    |
| Proposed        | —    | —          | —         | —    | —    |

## 19 Phase 15: Paper Writing Plan

The final research paper should have the following sections:

1. Abstract.
2. Introduction.
3. Related Work.



4. System Model.
5. LMS Parameter Adaptation.
6. LQR Nominal Controller.
7. Risk-Triggered Safety Filter.
8. Adaptive MPC Formulation.
9. Backup Safety Controller.
10. ROS2/Gazebo Experimental Setup.
11. Results and Discussion.
12. Limitations.
13. Conclusion.

## 20 Phase 16: Honest Limitations

Include these limitations in the paper:

- The LMS estimator requires sufficient excitation.
- Recursive feasibility is not guaranteed under sudden obstacle appearance.
- LiDAR-based obstacle extraction may introduce perception errors.
- The method is validated in Gazebo before real hardware.
- Dynamic obstacle prediction assumes short-horizon constant velocity.

Including limitations makes the paper more believable.

## 21 Phase 17: Publication Checklist

Before submitting, make sure you have:

1. Working proposed controller.
2. All baseline controllers tested.
3. At least 5 scenarios.
4. At least 5 repeated runs per scenario.
5. Clear plots.
6. One comparison table.
7. Clean mathematical formulation.
8. Honest discussion of limitations.
9. Proper references.
10. No exaggerated claims such as “guaranteed safety in all cases”.

## 22 Suggested Timeline

| Week    | Task                                                          |
|---------|---------------------------------------------------------------|
| Week 1  | Run existing LQR, MPC, adaptive MPC, and hybrid baselines.    |
| Week 2  | Implement LMS adaptation cleanup and command distortion node. |
| Week 3  | Implement LQR rollout risk predictor.                         |
| Week 4  | Implement adaptive MPC safety filter wrapper.                 |
| Week 5  | Add backup safety controller and logging.                     |
| Week 6  | Run Gazebo experiments and collect results.                   |
| Week 7  | Generate plots and comparison tables.                         |
| Week 8  | Write paper draft.                                            |
| Week 9  | Revise paper, fix weak experiments, improve figures.          |
| Week 10 | Submit to a suitable conference or workshop.                  |

## 23 Final Research Claim

The final paper should claim:

This work proposes a risk-triggered adaptive MPC safety filter for LQR-based differential-drive robot navigation. The method preserves the computational efficiency of LQR in safe regions while using adaptive MPC only when predicted safety violations are detected. Online LMS adaptation improves prediction under actuation mismatch, and a backup safety layer handles infeasible MPC cases.

## 24 What Not to Claim

Avoid these claims:

- Do not claim guaranteed publication.
- Do not claim global optimality.
- Do not claim safety under all possible obstacles.
- Do not claim real-world validation unless hardware experiments are actually done.
- Do not present command blending as the main proposed method.

## 25 Conclusion

This implementation plan converts the project from a weak hybrid switching idea into a stronger safety-filtering architecture. The key idea is simple: LQR should do normal tracking, while adaptive MPC should intervene only when the predicted LQR behavior becomes unsafe. This framing is easier to implement, easier to explain, and more defensible for a research paper.